Negative numbers and binary arithmetic

FD02 FACT FILE

Specification: students should be able to

- Perform binary addition of unsigned integers, showing carries
- Represent signed integers using sign and magnitude and explain its limitations
- Demonstrate how the two's complement system represents positive and negative numbers using 8 bits
- Convert between denary and 8-bit two's complement in both directions
- Perform addition and subtraction using two's complement
- Explain overflow and recognise when it occurs

Key words: binary addition, carry, signed integer, unsigned integer, sign bit, sign and magnitude, two's complement, invert (one's complement), negative place value, range, subtraction, overflow, status register (overflow flag), most significant bit (MSB)

Two's complement: the MSB is worth −128

	−128	64	32	16	8	4	2	1
+19	0	0	0	1	0	0	1	1
Invert every bit	1	1	1	0	1	1	0	0
Add 1 → −19	1	1	1	0	1	1	0	1

Check: −128 + 64 + 32 + 8 + 4 + 1 = −19 · Range in 8 bits: −128 to +127

Overflow

```
  01110000 (112)   Two positives gave a negative:
+ 00100000 ( 32)   the true answer 144 is
= 10010000 (reads as −112)   too big for +127, so overflow.
```

Forming -19 in 8-bit two's complement by inverting the bits of +19 and adding 1, and an example of overflow when two positive numbers give a negative result

Signed numbers in a fixed number of bits

Computers often need negative values – temperatures, bank balances, altitudes below sea level, changes in stock – but a binary register cannot hold a minus sign. Instead, one of the bits must carry the sign information and the whole system must agree how to interpret the pattern. Two schemes are examined. Sign and magnitude simply reserves the most significant bit as a flag. Two's complement gives the most significant bit a negative place value, which turns out to make arithmetic much simpler for hardware. Either way, the number of bits is fixed, usually eight in exam questions, and that fixes the range of values that can be stored. The same eight bits can be read in different ways: 10111001 is 185 as an unsigned integer, −57 in sign and magnitude and −71 in two's complement. This is why a question always tells you which representation to use, and why the answer must state it. Arithmetic on fixed-width numbers also introduces the risk of overflow, where the true answer lies outside the range. Understanding these ideas explains why programming languages offer data types such as byte, sbyte, int and long.

Binary addition

Binary addition follows the same column method as denary addition, but a carry happens as soon as a column total reaches two. The four rules are: 0 + 0 = 0; 0 + 1 = 1; 1 + 1 = 0 carry 1; and 1 + 1 + 1 = 1 carry 1. Work from the least significant bit on the right towards the left, writing any carry under the next column so it is included in that column's total. For example, adding 01011011 and 00110110 produces carries in six columns and gives 10010001, which is 145 – the same as 91 + 54. Always check the answer by converting each value to denary. Adding 1 to a run of 1s makes the carry ripple through every bit, which is why 0111 + 1 becomes 1000. When the numbers are unsigned, a carry out of the most significant bit means the result needs more bits than are available, which is overflow. When the numbers are in two's complement, the same circuit is used but the rule for detecting overflow changes, as explained below.

Sign and magnitude

In sign and magnitude the most significant bit is used only to show the sign, 0 for positive and 1 for negative, and the remaining bits give the size of the number in ordinary binary. So +19 is 00010011 and −19 is 10010011. It is easy for people to read, because the magnitude bits are identical for the positive and negative versions. However, it has two serious weaknesses. First, there are two representations of zero, 00000000 and 10000000, which wastes a pattern and means a comparison with zero must check two values. Second, and more importantly, ordinary binary addition gives wrong answers when the signs differ: 5 + (−3) worked as 00000101 + 10000011 gives 10001000, which reads as −8 rather than +2. A processor would need extra logic to compare signs and magnitudes and choose whether to add or subtract. Because one bit is used for the sign, n bits give a range of $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$, which is −127 to +127 for a byte.

Two's complement

In two's complement the most significant bit has a negative place value; in eight bits the columns are −128, 64, 32, 16, 8, 4, 2 and 1. Positive numbers look exactly as they do in unsigned binary and always start with 0; any pattern starting with 1 is negative. To convert a negative denary number, write the positive value in binary, pad it with leading zeros to eight bits, invert every bit and then add 1. To convert from two's complement to denary, add up the place values of the 1s, remembering that the first column is −128. There is only one zero, and the range for n bits is -2^{n-1} to $+(2^{n-1} - 1)$, so a byte holds −128 to +127. A useful check is that a number added to its two's complement gives eight zeros once the carry out is ignored. The great advantage is that subtraction becomes addition: A − B is calculated as A plus the two's complement of B, so the same adder circuit handles both operations and no sign-comparison logic is needed. This is why virtually all modern processors use two's complement for integers.

Subtraction and the discarded carry

To subtract in two's complement, first form the negative of the number being subtracted, then add. For 45 − 19, the two's complement of 19 is 11101101, and 00101101 + 11101101 gives a ninth bit: 1 00011010. Because the register only holds eight bits, the carry out of the most significant bit is discarded, leaving 00011010, which is 26, the correct answer. When the answer is negative, as in 38 − 52, there is no carry out: 00100110 + 11001100 gives 11110010, whose leading 1 tells you it is negative, and the place values give −128 + 64 + 32 + 16 + 2 = −14. Pupils often think the discarded carry is an error, but in two's complement arithmetic it is a normal part of the process whenever a positive and a negative number are added and the result is positive. It does not, by itself, indicate overflow. The correctness of the eight-bit result is decided by the signs of the inputs and output, not by whether a carry left the register.

Overflow

Overflow occurs when the true result of a calculation cannot be represented in the number of bits available. The test depends on the representation. For unsigned numbers, a carry out of the most significant bit means overflow: 200 + 100 = 300, which needs nine bits, so the eight-bit result 00101100 reads as 44. For two's complement, overflow happens only when two numbers with the same sign are added and the result has the opposite sign. Adding 100 and 50 gives 10010110, which reads as −106 because +150 is beyond +127; adding −100 and −50 gives 01101010, which reads as +106 because −150 is below −128. Adding a positive and a negative number can never overflow, since the result always lies between them. Processors record overflow in a flag in the status register, and well-written programs check it or use a larger data type such as a 32-bit int. If overflow goes undetected, the program carries on with a wrong value, which can produce incorrect output, a crash or, in control systems, dangerous behaviour.

Key terms from the mark schemes

Binary addition
Adding binary numbers column by column from the right using the rules 0+0=0, 0+1=1, 1+1=0 carry 1, 1+1+1=1 carry 1.

Carry
A bit passed to the next column on the left when a column total is two or more.

Sign bit
The most significant bit of a signed number, used to show whether the value is positive or negative.

Sign and magnitude
A signed-integer representation in which the MSB indicates the sign (0 positive, 1 negative) and the remaining bits give the size.
`Credited: sign-magnitude`

Two's complement
A signed-integer representation in which the MSB has a negative place value; a number is negated by inverting its bits and adding 1.

Range (integers)
The smallest and largest values a representation can store with a given number of bits, e.g. −128 to +127 in 8-bit two's complement.

Overflow flag
A bit in the processor's status register that is set when an arithmetic operation overflows.

Unsigned integer
A whole number stored with no sign bit, so all bits represent magnitude and only zero or positive values are possible.

Overflow
An error that occurs when the result of a calculation is too large or too small to be stored in the available number of bits.

C C# example

Two's complement to denary

```
byte raw = 0b1011_1001;               // the stored 8 bits
sbyte value = unchecked((sbyte)raw);   // read them as two's complement
Console.WriteLine(raw);                // 185 if read as unsigned
Console.WriteLine(value);              // -71 if read as signed
```

Exam focus

Two's complement conversions usually carry a mark for the method and a mark for the answer, so show the positive binary padded to eight bits, the inverted bits and the result of adding 1, or show the place-value sum including −128. Check your answer by converting back; this also catches the common slip of forgetting to add 1. In addition questions, write the carry bits clearly beneath the sum and convert the result to denary when asked. For subtraction, state that you are adding the two's complement of the second number and say that the carry out of the most significant bit is discarded. When asked whether overflow has occurred, answer yes or no and justify it with numbers, for example 'the true answer is 150, which is greater than the largest value, 127, so overflow has occurred'. State ranges with both ends, such as −128 to +127, never −128 to +128. 'Why is two's complement preferred?' needs reasons such as a single zero and the ability to subtract using an adder, each briefly explained.

? Check yourself

1. State the range of integers that can be represented in 8-bit two's complement. [1]
2. Represent −45 in 8-bit sign and magnitude. [1]
3. Represent −45 in 8-bit two's complement. Show your working. [2]

Answers: 1. −128 to +127 (1) 2. 10101101 (1) 3. 45 = 00101101, invert to 11010010 (1) / add 1 = 11010011 (1)